

libbpf-bootstrap is essentially a collection of sample eBPF programs with all the setup code and a Makefile (which are tedious to write, hard to get right, and distracting). The idea is that one could pick a suitable example as a playground or a template for a serious project (sticking to the licence, which is BSD 3-Clause at the time of writing this).

The most minimal example in this collection is a program called *minimal*. Let's see how to build and execute it.

First, make sure your system has git, make, gcc, clang, and libelf-dev. Then run the following commands:

```
1. git clone --recurse-submodules https://github.com/libbpf/
   libbpf-bootstrap
2. cd libbpf-bootstrap/examples/c
3. make minimal
4. sudo ./minimal
```

When we did `sudo ./minimal`, we just started the user space component that injects the actual eBPF program into the kernel. This should print a success message and wait. We can open a new terminal and enter the following command to see the output from the eBPF program:

```
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

This will keep printing 'bpf_trace_printk: BPF triggered from PID ...'. Killing the loader program by pressing `Ctrl+C` in the first terminal will detach the eBPF program from the kernel, at which point there will be no new logs in the second terminal (unless some other program is printing to it).

Alright, we loaded an eBPF program, and it kept printing a message every second. What is happening here?

Let's check the relevant part from *minimal.bpf.c*, the eBPF program:

```
SEC("tp/syscalls/sys_enter_write")
int handle_tp(void *ctx)
{
    ...
    bpf_printk("BPF triggered from PID %d.\n", pid);
    return 0;
}
```

The first line says that the eBPF program is to be triggered whenever the `write()` system call gets called. `write()` is what makes high-level output functions like `printf()` and `fwrite()` possible. So, this eBPF program has the potential to trace all such output operations by any process (except that the example from libbpf-bootstrap repo contains a check to filter out the ones from sources other than our loader).

Now what does the loader program (*minimal.c*) contain? After all the setup code, it contains this:

```
for (;;) {
    /* trigger our BPF program */
    fprintf(stderr, ".");
    sleep(1);
}
```

That is, it prints a dot every second, which would trigger the eBPF program as discussed above.

For an exercise, make the following changes to *minimal.bpf.c* and redo everything from *make minimal* to loading the programs and observing the logs:

1. Change SEC("tp/syscalls/sys_enter_write") to SEC("tp/syscalls/sys_enter_rmdir").
2. Remove the pid check so that `rmdir()` issued by any process will be handled.

Once the program is loaded and you are observing the output, create an empty directory and remove it (again, either using GUI or the commands). You can see that the removal of the directory is being logged by our eBPF program. Well, our bpftrace attempt was much simpler than this, but remember that here the possibilities are virtually unlimited compared to bpftrace.

Here are three tools that could make eBPF development and management easier:

- **bptop**: Real-time view of running eBPF programs, like with `top` and `htop`.
- **bpftrace**: Inspection and manipulation of eBPF programs and maps.
- **llvm-objdump**: Disassemble eBPF bytecode (so that you can inspect the Assembly code).

More to explore

There are many layers to work at if you are enthusiastic about eBPF. You can make use of technologies built on top of eBPF (as a sysadmin), you can develop tools that generate or manipulate eBPF programs (as a compiler engineer), or you can even work on the eBPF-related parts of the kernel, especially the verifier. Potential loopholes in the verification are always a concern. At the same time, false alarms resulting from stricter verification can make eBPF development a frustrating experience. This is why many people are interested in improving the verifier.

Now, make sure to visit ebpf.io and ebpf.foundation, where you can learn more about eBPF, its applications, and opportunities. 

By: Nandakumar Edamana

The author has been involved with free software since his school days. A contributor to mainstream media since 2013, he has published three books in Malayalam. He was a software architect prior to joining one of the IITs as a PhD student.